

Colored Cube Recognition

ROS 2 • YOLOv5 • TensorRT

REAL-TIME COMPUTER VISION

Concept Illustration

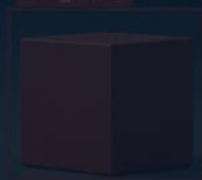
Colored Cube Detection on NVIDIA Jetson Hardware

YOLO • TensorRT • ROS 2

SEE
UNDERSTAND
MOVE
EXPLORE



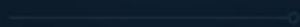
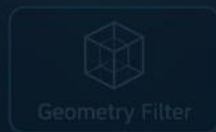
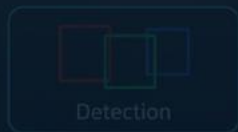
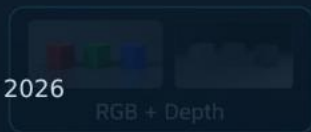
red_cube



green_cube



blue_cube



Maher | 13 September 2026

The problem and what success means

02

Find real red, green and blue cubes in a live indoor camera view and provide their locations to robot software.

Detect the right objects

Correct color and box; ignore ordinary background objects.

Respond as the scene changes

Run on the onboard Jetson and check behavior across positions and distances.

Provide useful locations

Distinguish image position from physical distance; verify the depth-based output.

Intended range: 20–80 cm. Scope: perception; navigation and picking are outside this project.

Computer vision: what should the system see?

03

Computer vision turns image pixels into useful information.

What is it?

A red, green or blue cube.

Where is it in the image?

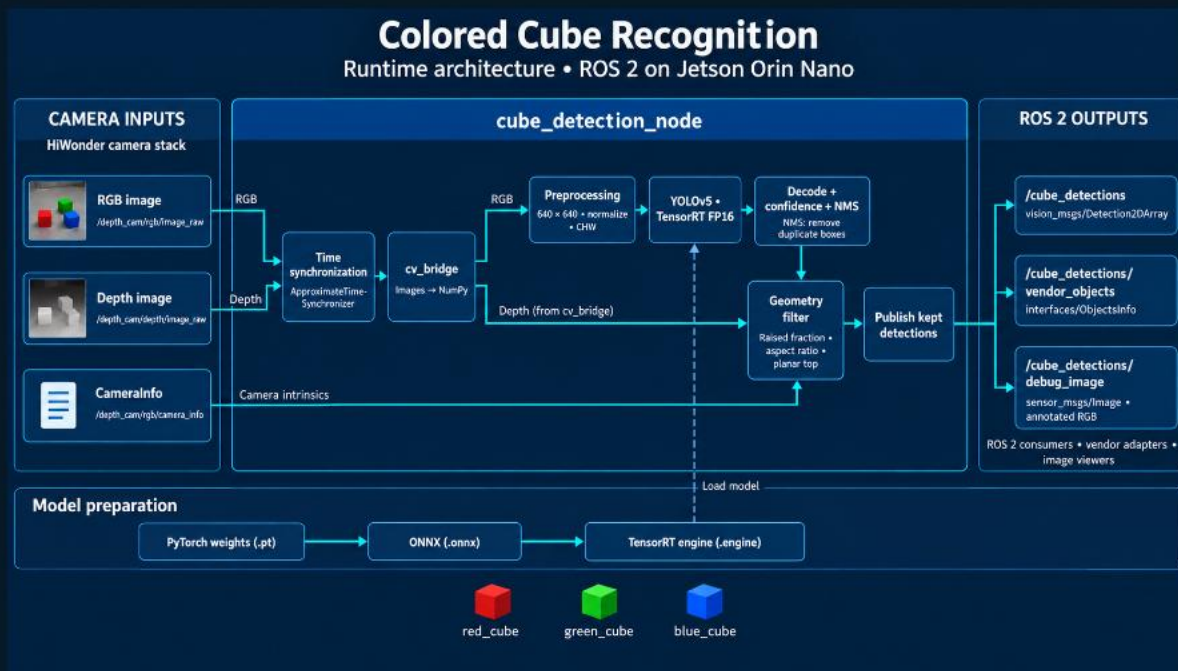
A bounding box marks its location.

How far away is it?

Depth data adds physical distance.

System architecture

04



Why this approach?

05

Color thresholds **A simple starting point**

They isolate colored regions, but color alone does not establish that an object is a cube.

Pretrained YOLO + fine-tuning **Our main detector**

Learn cube examples from our camera while reusing visual features and the existing Jetson runtime.

Depth / geometry check **A supporting stage**

Add a physical-shape check after detection to reject unsuitable candidates.

Design rationale, not a benchmark ranking of competing detectors.

My contribution and the components I reused

06

My engineering work

Arrange and capture scenes
Review cube boxes and colors
Run training and deployment
Interpret validation and live tests

Existing components

Vendor camera infrastructure
Pretrained YOLO model
PyTorch training library
TensorRT and ROS 2

Codex assistance

Prepare and adapt code
Guide the technical steps
Help check outputs
Prepare documentation

The project work is connecting these parts, making justified choices, and checking what actually works.

Data quality: useful examples and honest checks

07

Vary the scene

Change cube position, distance and arrangement. Include ordinary objects and empty scenes.

Review the expected answers

A human-approved color and box define each cube. Check the labels before exporting them.

Separate whole arrangements

Do not split almost identical neighboring frames between learning and validation.

Research decision: validation uses new arrangements. One room and limited lighting still restrict coverage.

Training workflow: learning from examples

08



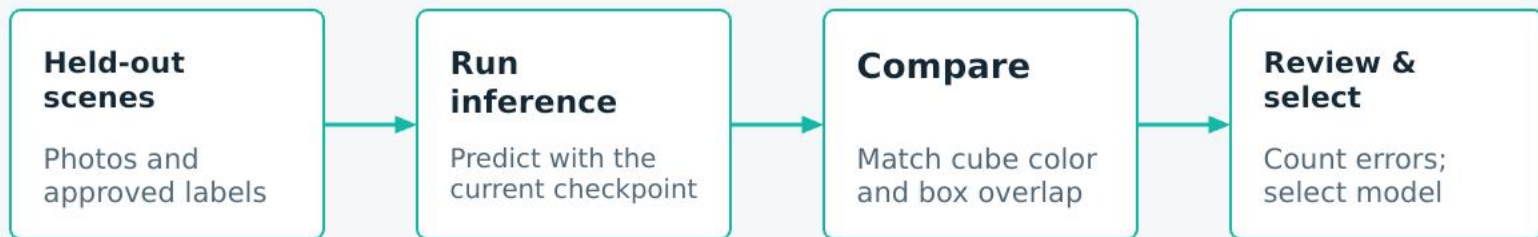
Inside fine-tuning

Predict → Compare with labels → Adjust weights → Repeat

Weights = learned numbers. Pretrained = learned earlier.
Fine-tuning adapts them; a checkpoint saves their current state.

Validation workflow: checking new scenes

09



Correct match

Right color and a matching location.

Extra detection

A predicted box with no matching cube.

Missed cube

A real cube without a matching prediction.

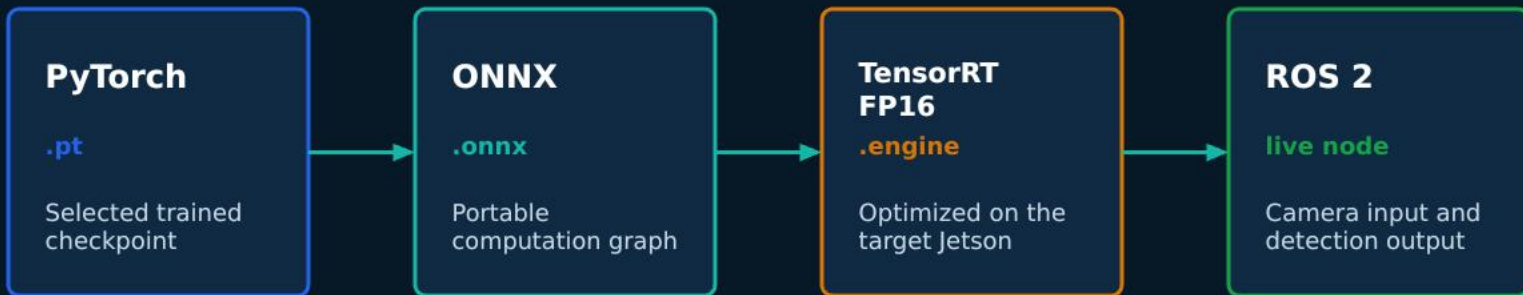
Validation does not update weights. After model selection, test the complete system live.

Deployment on NVIDIA Jetson

10

DESKTOP · train and export

JETSON · optimize and run



**The desktop learns the weights. The Jetson uses them to predict.
Live inference does not retrain the model.**

Why add a geometry filter?

11

A colored region can look convincing without being a cube.



Two requirements: reject distractors and retain real cubes.

Compare the same scene with the filter off and on to isolate its effect.

Dataset and training outcomes

12

23

training images

8

validation images

45

best-checkpoint epoch

Balanced color coverage

13 cubes per color in training.
6 per color in validation.
Background-only images in both sets.

One completed fine-tuning run

60 passes through the training set.
Validation selected the checkpoint.
Data still comes from one room.

Validation result: one missed red cube

13



Approved



Prediction

17 / 18

visible cubes
detected

1

false negative
red cube in V03

Across all eight validation images

17 visible matches · 1 missed cube · 0 additional boxes

Visual review of saved predictions; detailed automated metrics are in the appendix.

Live results on the Jetson

14

/cube_detections/debug_image



Score 0.78 = model confidence, not 78% accuracy.

Three colors together

Three correct visible boxes.

Empty room scene

No visible detection.

Processing time

About 58-59 ms per frame*

*Median node processing in observed windows. Filter off; confidence cutoff 0.25. Brief scene checks.

Geometry filter limitation

15

≈ 4

model candidates per
frame

0

detections kept

flat

only rejection reason

The detector continued to produce candidates. The downstream depth-based filter removed every candidate, including the real cubes.

Rejection stage identified. Root cause still needs depth and alignment checks.

Conclusion: demonstrated behavior and open work

16

Demonstrated

The fine-tuned model runs on Jetson and detected all three colors in the observed live scene.

Limited evidence

Background and runtime checks are brief. Full-range and broader-scene reliability remain unmeasured.

Unfinished

The geometry filter rejects valid cubes. Reliable depth-based localization still needs verification.

Next: capture aligned image/depth evidence, diagnose the filter, then repeat the full-system tests.

Appendix: understanding the validation scores

17

Precision · 0.982

How many reported detections are correct?

Recall · 0.964

How many real cubes are found?

mAP50 · 0.995

Detection quality across confidence levels at 50% box overlap.

mAP50-95 · 0.707

Average precision across increasingly strict box-overlap requirements.

Why 17/18 differs: the visible-box count uses a display cutoff; the library reports metrics using its evaluation curves and selected operating point.

Appendix: research and experiment references

18

Research basis

PyTorch: transfer learning
scikit-learn: grouped evaluation
Dive into Deep Learning: generalization

Project decisions

Training/validation methodology review
Indoor capture checklist
Architecture and technical walkthrough

Reproducible run

YOLOv5u-small · 23 training / 8 validation images
60 epochs · batch 4 · 640-pixel input
AdamW · learning rate 0.001 · seed 42

Sources, run identity and settings are recorded in the speaker notes.